

SwapUrMusic

Convierte y comparte tus canciones y listas de reproducción al instante

Match and swap your music links

■ <https://swapurmusic-khuqyu7mpssvyvpjxay4r.streamlit.app/>

Aplicación Web con IA

Streaming Musical

Chatbot de búsqueda

Propuesta de Proyecto

ESTUDIANTE

Santiago Suarez Foti

COMISIÓN

86425

MATERIA

**IA: Prompt Engineering para
programadores**

INSTITUCIÓN

**Diplomatura Data Science —
Coderhouse**

Propuesta desarrollada para la materia Inteligencia Artificial: Prompt Engineering para programadores (plataforma-beta.coderhouse.com/cursos/86425/prompt-engineering-dev-carreras-intensivas) de la Diplomatura en Data Science de Coderhouse. La app SwapUrMusic ya se encuentra desplegada y en funcionamiento.

Objetivos del proyecto

- Proponer una aplicación web que resuelva un problema real del ecosistema musical digital.
- Presentar una propuesta que describa cómo llevar a cabo el proyecto con herramientas de IA.
- Comprender el concepto de prompt y su utilización en la generación de código mediante IA.
- Considerar la factibilidad técnica y económica del proyecto.
- Dar cuenta de la instalación y configuración de las herramientas del curso.
-

Problemática: fragmentación del ecosistema musical

DESCRIPCIÓN

Actualmente los usuarios consumen música en múltiples plataformas de streaming: Spotify, YouTube Music y Deezer, entre otras. Cuando alguien comparte un link de una canción o playlist, el destinatario puede no tener acceso a esa plataforma, perdiéndose la experiencia musical o debiendo buscar manualmente el contenido en su servicio preferido.

SITUACIONES CONCRETAS

- **Link incompatible:** Un amigo comparte un link de Spotify pero el destinatario usa YouTube Music y no puede abrirlo directamente.
- **Playlists inaccesibles:** Transferir una playlist entre plataformas de forma manual es tedioso y muy propenso a errores.
- **Versiones incorrectas:** Buscar cada canción a mano implica perder tiempo y encontrar versiones equivocadas: remixes, ediciones TV, versiones cortas.
- **Sin asistencia para buscar:** No existe una herramienta gratuita que ayude al usuario a buscar una canción y obtener sus links en todas las plataformas al instante.

LIMITACIONES DE LAS SOLUCIONES ACTUALES

Herramientas como Soundiiz o TuneMyMusic son pagas y requieren registro obligatorio. Ninguna ofrece matching inteligente de versiones ni un asistente para buscar canciones y obtener sus links en todas las plataformas en un solo paso.

Solución propuesta

¿QUÉ RESUELVE?

SwapUrMusic permite pegar el link de una canción o playlist desde Spotify, YouTube/YouTube Music o Deezer, y obtener instantáneamente los links equivalentes en las otras dos plataformas. Además incluye un chatbot donde el usuario puede escribir el nombre de una canción y recibir los links directos en Spotify, YouTube y Deezer al instante, sin registro y gratuitamente.

¿POR QUÉ ES RELEVANTE?

- **Inmediatez:** links destino en segundos, sin pasos intermedios ni cuentas requeridas.
- **Accesibilidad:** completamente gratuita, sin registro. Funciona desde cualquier navegador, incluso en celular.
- **Matching inteligente:** combina fuzzy matching (RapidFuzz/Levenshtein) y similitud semántica (Gemini IA) para encontrar la versión más precisa disponible.
- **Fallback automático:** si no encuentra la canción exacta, genera un link de búsqueda directo en la plataforma destino para que el usuario la encuentre fácilmente.
- **Chatbot de búsqueda:** el usuario escribe el nombre de una canción y el chatbot devuelve los links directos en Spotify, YouTube y Deezer.

PLATAFORMAS SOPORTADAS

Spotify	YouTube / YouTube Music	Deezer
---------	-------------------------	--------

SwapUrMusic — Propuesta de aplicación web con IA

NOMBRE Y FUNCIÓN PRINCIPAL

SwapUrMusic es una aplicación web construida en Python con Streamlit. Detecta automáticamente si el link ingresado es una canción individual o una playlist, extrae los metadatos (artista, título, versión) y busca el equivalente en las plataformas destino usando scraping público y APIs, con un motor de matching con IA.

CÓMO SE INTEGRA LA IA

- **Match exacto (■ Coincidencia Exacta):** misma canción, mismo artista, misma versión → link directo de alta confianza.
 - **Match alternativo (■ Coincidencia Cercana):** misma canción, versión diferente → link con aviso. Score calculado con RapidFuzz + Gemini IA.
 - **Fallback (■ Búsqueda Directa):** sin resultado exacto → link de búsqueda automático en la plataforma destino.
 - **Chatbot de búsqueda:** el usuario escribe el nombre de una canción y recibe los links directos en Spotify, YouTube y Deezer. Funciona vía OpenRouter con modelos gratuitos.
- Limitación conocida: las playlists de Spotify no pueden convertirse porque la API de Spotify Developer (plan gratuito) requiere autenticación del usuario para acceder a playlists. Las canciones individuales y playlists de YouTube y Deezer funcionan correctamente.

VENTAJAS Y BENEFICIOS

Gratuito	Sin registro	Logo animado	Chatbot búsqueda	Matching con IA
----------	--------------	--------------	------------------	-----------------

Arquitectura técnica de la aplicación

ESTRUCTURA MODULAR DEL CÓDIGO

- **main.py:** Entrada principal. Renderiza la UI en Streamlit con tema oscuro, logo animado y animación en el título. Gestiona el flujo de conversión y el chatbot.
- **services/spotify_service.py:** Extracción de metadatos e información pública de canciones y búsqueda en Spotify vía API Developer (plan gratuito). No accede a playlists privadas del usuario.
- **services/youtube_service.py:** Scraping público de YouTube usando oEmbed y extracción de datos del inicializador público de playlists (ytInitialData). No usa API de YouTube.
- **services/deezer_service.py:** Búsqueda directa por API pública de Deezer. No requiere autenticación ni API key.
- **utils/matcher.py:** Motor de matching: fuzzy matching (RapidFuzz/Levenshtein) + similitud semántica con Gemini IA (opcional). Penaliza versiones de video, ediciones TV y versiones cortas.

FLUJO DE FUNCIONAMIENTO

Paso	Acción	Tecnología
1	Usuario pega un link de canción o playlist	Streamlit UI
2	Detección automática de plataforma y tipo (track/playlist)	Regex + URL parser
3	Extracción de metadatos (artista, título, versión)	Spotify API / YouTube scraping / Deezer API
4	Búsqueda en las otras dos plataformas destino	APIs públicas + scraping
5	Cálculo de score de coincidencia	RapidFuzz + Gemini IA (si está configurado)
6	Resultado con nivel de confianza visual	Tarjetas Streamlit (■ ■ ■)
7	Chatbot: usuario escribe canción → recibe links en 3 plataformas	OpenRouter API (free tier)

Prompt inicial y desarrollo con IA

PROMPT INICIAL — GENERACIÓN DE LA APP CON GOOGLE AI STUDIO

```
Contexto y Objetivo: Crea una aplicación web multi-plataforma y responsiva en Python utilizando Streamlit llamada 'SwapUrMusic'. El objetivo es permitir a los usuarios pegar enlaces de canciones o playlists de Spotify, YouTube/YouTube Music o Deezer, y convertirlos automáticamente a las otras dos plataformas. Estructura Modular: main.py – Entrada principal y UI en Streamlit. /services/spotify_service.py – Extracción de info pública y búsqueda en Spotify (API Developer). /services/youtube_service.py – Scraping público oEmbed + ytInitialData (sin API de YouTube). /services/deezer_service.py – Búsqueda por API pública de Deezer (sin autenticación). /utils/matcher.py – Matching inteligente con RapidFuzz/Levenshtein + Gemini IA. Requisitos clave: - Funcionar en modo scraping público si no hay API keys configuradas. - Detección automática de track vs playlist. - Fallback con link de búsqueda si no se encuentra la canción. - Score de confianza visible: Exacta (verde), Cercana (amarillo), Búsqueda (azul). - UI dark premium 'Cosmic Twilight' (#0f111a) con gradiente púrpura-azul en el título. - Tarjeta premium para canción única con badge de precisión del match. - Diseño centrado y minimalista, sin sidebar clutter.
```

CÓMO SE ELIGIÓ Y USÓ ESTE PROMPT

El prompt describe con precisión la arquitectura modular, el comportamiento esperado en cada caso (match exacto, alternativo, fallback), el diseño visual y las restricciones técnicas de cada API. Esta especificidad permite que Google AI Studio genere un código base funcional desde la primera iteración.

EXTENSIÓN DEL PROYECTO — VISUAL STUDIO CODE + CLAUDE CODE + OPENROUTER

Una vez generada la app base con Google AI Studio, el proyecto fue descargado y extendido localmente usando Visual Studio Code con el plugin Claude Code conectado a la API gratuita de OpenRouter (openrouter/free), junto con el plugin Superpowers y skills de la comunidad. En esta etapa se agregaron al proyecto:

- **Chatbot de búsqueda:** el usuario escribe el nombre de una canción y recibe los links en Spotify, YouTube y Deezer al instante.
- **Logo personalizado:** diseño de logo propio integrado en la interfaz.
- **Animación en logo y título:** animación CSS aplicada al logo y al título de la app para una experiencia visual premium.
- **Mejora en la búsqueda de links:** refinamiento del algoritmo de matching para mayor precisión en los resultados.

Nota: no se usaron prompts adicionales en el sentido tradicional. El trabajo de extensión se realizó de forma iterativa directamente en el editor, usando Claude Code como asistente de código en tiempo real.

Factibilidad económica

COSTOS DEL PROYECTO

Componente	Herramienta / Servicio	Costo
Generación de código base	Google AI Studio	\$0 — cuota gratuita
Extensión con chatbot y logo	VS Code + Claude Code + OpenRouter	\$0 — free tier
Plugin de asistencia en IDE	Superpowers + skills comunidad	\$0 — gratuito
Motor de matching local	RapidFuzz / Levenshtein (Python)	\$0 — open source
Matching semántico avanzado	Gemini API (opcional)	\$0 — cuota gratuita
Chatbot de búsqueda	OpenRouter (modelos free tier)	\$0 — gratuito
API Spotify (info pública)	Spotify Developer (plan gratuito)	\$0 — sin costo
YouTube (datos públicos)	Scraping oEmbed / ytInitialData	\$0 — info pública
API Deezer	API pública sin autenticación	\$0 — sin key
Deploy y hosting	Streamlit Community Cloud	\$0 — plan gratuito
COSTO TOTAL DEL PROYECTO		\$0 — 100% gratuito

LIMITACIONES DEL MODELO Y CONSIDERACIONES

- Alucinaciones del chatbot:** al usar modelos de IA para buscar canciones, puede devolver links incorrectos o inventados. Se recomienda verificar los resultados.
- Dependencia de APIs externas:** cambios en las APIs de Spotify, YouTube o Deezer podrían afectar el funcionamiento de la app.
- Playlists de Spotify:** no disponibles sin autenticación del usuario (limitación de la API Developer gratuita de Spotify).

Requerimientos técnicos

Completar con capturas de pantalla o GIF según se indica en cada ítem

HERRAMIENTAS INSTALADAS Y CONFIGURADAS

Streamlit Community Cloud App desplegada en https://swapurmusic-khuqyu7mpssvyvpoxay4r.streamlit.app/ — Agregar captura de pantalla del inicio de sesión con el nombre de usuario de la cuenta en streamlit.io. Activo
Google AI Studio Herramienta de IA utilizada para la generación del código base de la app mediante el prompt inicial. Agregar captura de pantalla de Google AI Studio activo con el proyecto cargado. Usado
Visual Studio Code + Claude Code Editor usado para extender el proyecto descargado. Claude Code conectado vía OpenRouter/free. Agregar GIF o video mostrando Claude Code funcionando en VS Code. Instalado
Plugin Superpowers + Skills Plugins de la comunidad instalados en VS Code que potencian a Claude Code con skills específicas. Agregar captura de pantalla de los plugins instalados en VS Code. Instalado
OpenRouter API (free tier) API usada por Claude Code y por el chatbot integrado de la app. Plan gratuito, sin costo. Agregar captura de la API key generada en openrouter.ai. Configurado
Spotify Developer Dashboard Credenciales Client ID y Client Secret del plan gratuito para acceder a info pública de canciones. Agregar captura del dashboard en developer.spotify.com. Configurado

STACK TECNOLÓGICO COMPLETO

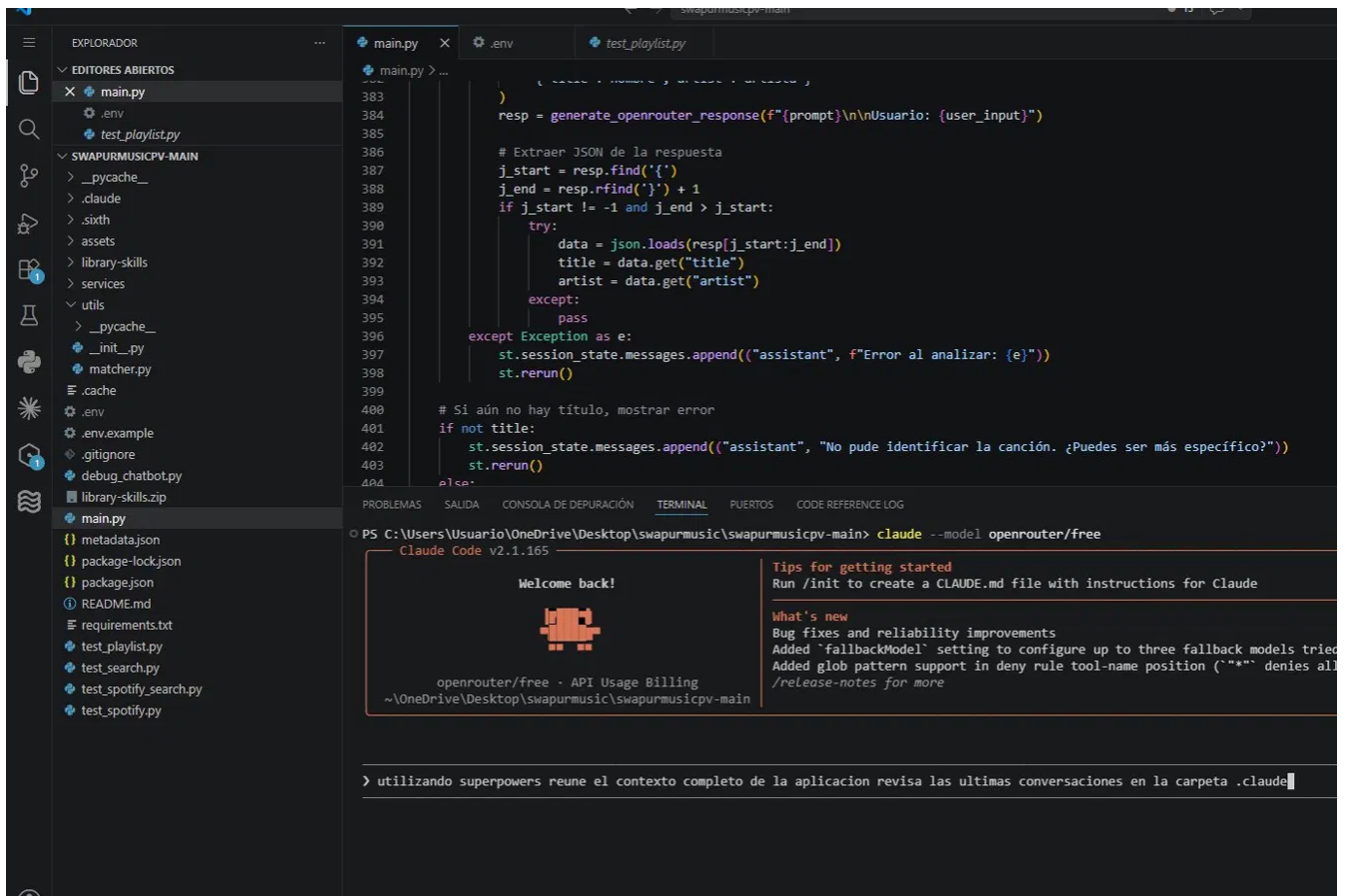
Capa	Tecnología	Rol en el proyecto
Frontend / UI	Streamlit (Python)	Interfaz web oscura con logo animado
Generación inicial	Google AI Studio	Código base generado con IA
Extensión / mejoras	VS Code + Claude Code + OpenRouter	Chatbot, logo, animaciones, mejoras
Matching IA	RapidFuzz + Gemini (opcional)	Motor de similitud para el matching

Chatbot búsqueda	OpenRouter API (free)	El usuario busca canciones por nombre
Spotify	Spotify Developer API (gratuita)	Info pública de canciones y búsqueda
YouTube	Scraping público (oEmbed/ytInitial)	Extracción sin API oficial
Deezer	Deezer API pública	Búsqueda sin autenticación
Deploy	Streamlit Community Cloud	Hosting gratuito de la app

SLIDE 9 — Requerimientos Técnicos: Evidencia Visual

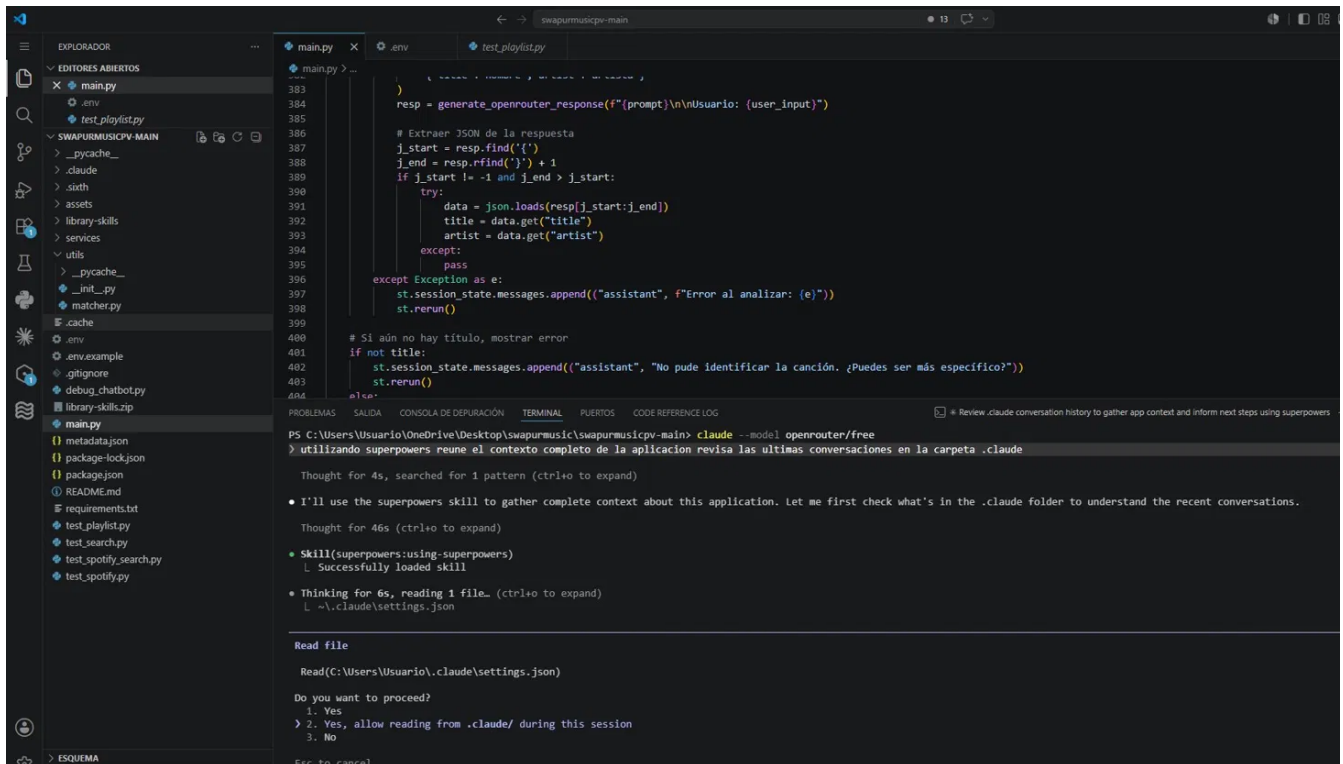
A continuación se presentan capturas de pantalla del entorno de trabajo configurado en Visual Studio Code con Claude Code y el plugin Superpowers, tal como fue utilizado para el desarrollo y extensión del proyecto SwapUrMusic.

■ Visual Studio Code + Claude Code (Claude v2.1.165) activo



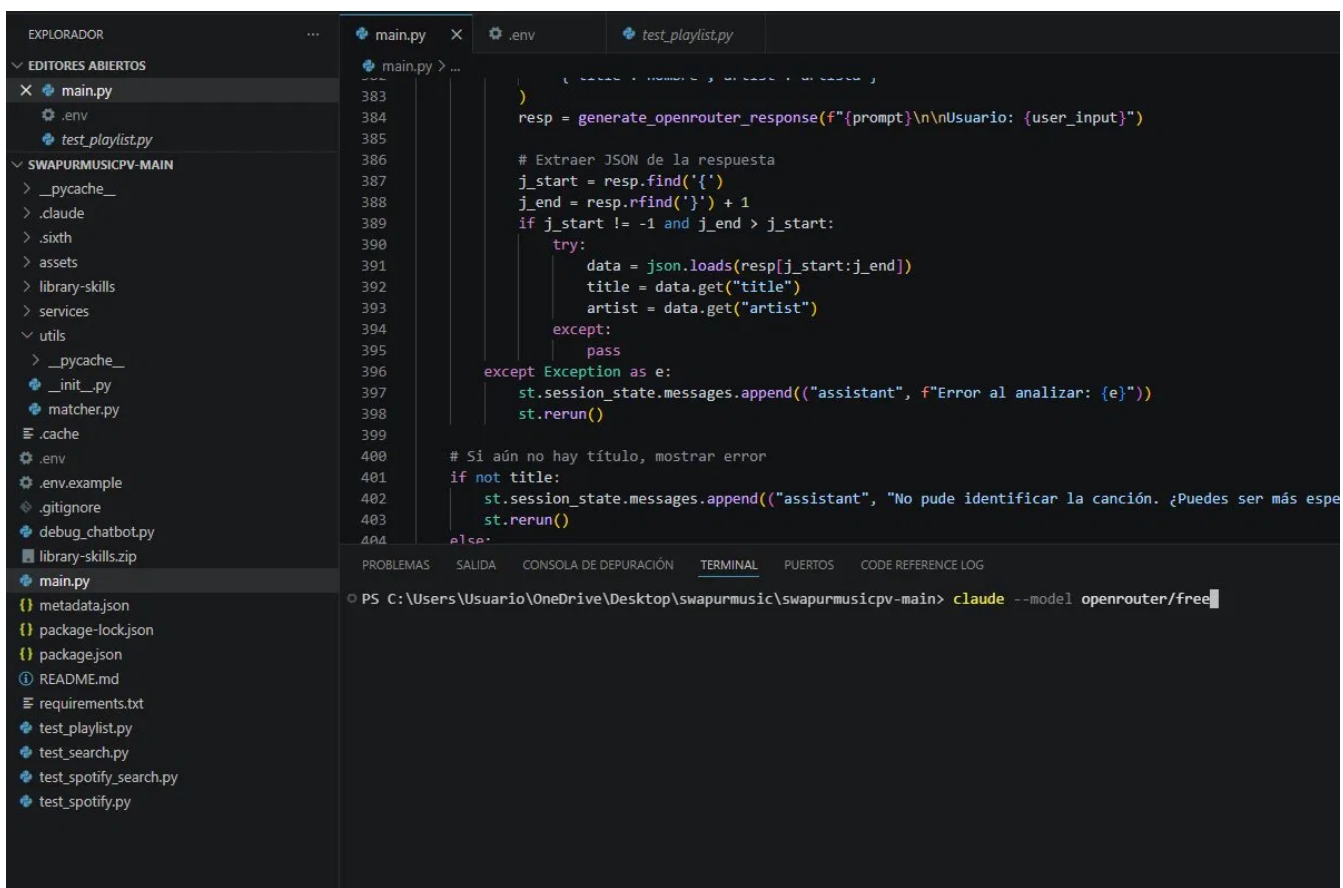
Claude Code v2.1.165 ejecutándose en el terminal integrado de VS Code, conectado a openrouter/free. Se observa el archivo main.py del proyecto SwapUrMusic abierto y el comando utilizado para iniciar el asistente de código.

■ Superpowers Plugin en acción — lectura de contexto del proyecto



El plugin Superpowers ejecutando el skill de análisis de contexto: Claude Code revisa las conversaciones en la carpeta `.claude` y lee los archivos del proyecto para generar una respuesta contextualizada. Se aprecia el permiso de acceso a `~/.claude/settings.json`.

■ Entorno completo: VS Code + Claude Code + OpenRouter/free



Vista general del entorno de desarrollo con `main.py` del proyecto `swapurmusicpv-main` abierto, terminal integrado mostrando Claude Code conectado a `openrouter/free` y el comando de consulta al asistente en tiempo real.

LINKS DEL PROYECTO

■ App desplegada en Streamlit Community Cloud:

<https://swapurmusic-khuqyu7mpssvyvpjxay4r.streamlit.app/>

■ Video demostrativo probando la app:

https://drive.google.com/file/d/10dpDcmduTpQx625-gh4C_DZC77TN9PIN/view?usp=sharing

■ Repositorio del proyecto en GitHub:

<https://github.com/santisf/swapurmusic.git>